



GRADUATION PROJECT

DermaVision

AI-assisted dermatology platform for preliminary skin image analysis, safe medical explanation, and consultation reports.

Project Overview

DermaVision allows users to upload a skin image, receive an AI-based preliminary diagnosis, ask diagnosis-aware questions through a safe RAG assistant, and generate printable reports for educational and consultation purposes.

FastAPI Backend

React/Vite Frontend

EfficientNet-B4

RAG Assistant

Arabic/English

How the Project Will Be Explained

1. Product Flow

User journey from signup to diagnosis, chat, history, and reports.

2. Backend

FastAPI, architecture, design patterns, diagnosis pipeline, database, RAG, and security.

3. Frontend

React/Vite structure, pages, localization, report UI, chat UI, and API integration.

4. AI and RAG

Model predicts the disease; RAG explains it using knowledge-base grounding.

5. Safety

Medical guardrails, no personal prescription, urgent-care detection, and validation.

6. Deployment

Docker, GitHub readiness, environment variables, and production notes.

Main User Flow

1

Sign up / Login

User creates an account and receives a JWT token for protected requests.

2

Upload Image

User uploads a JPG, JPEG, or PNG skin image from the diagnosis page.

3

Validate Image

Backend validates MIME type, size, extension, and real image content.

4

AI Prediction

EfficientNet-B4 predicts disease label, confidence, top predictions, and image quality.

5

Store Result

Diagnosis is stored with user ownership, image path, predictions, and timestamps.

6

View Report

Result page shows diagnosis, confidence, image quality, disease details, and disclaimer.

7

Ask Assistant

User can ask RAG questions about danger, contagion, symptoms, care, or treatment guidance.

8

Generate Reports

User can download a preliminary report or generate a consultation report after chat.

Why We Chose FastAPI

API-first and ML-friendly

The project is mainly a REST API around image upload, ML inference, auth, history, and RAG. FastAPI fits this style naturally.

Pydantic Validation

Request and response schemas protect the API from invalid data and make contracts clear for the frontend.

Why not Flask?

Flask is simple, but FastAPI gives typing, validation, dependency injection, and docs with less custom work.

Automatic API Docs

Swagger UI at `/docs` helps test endpoints during development and viva discussion.

Async Support

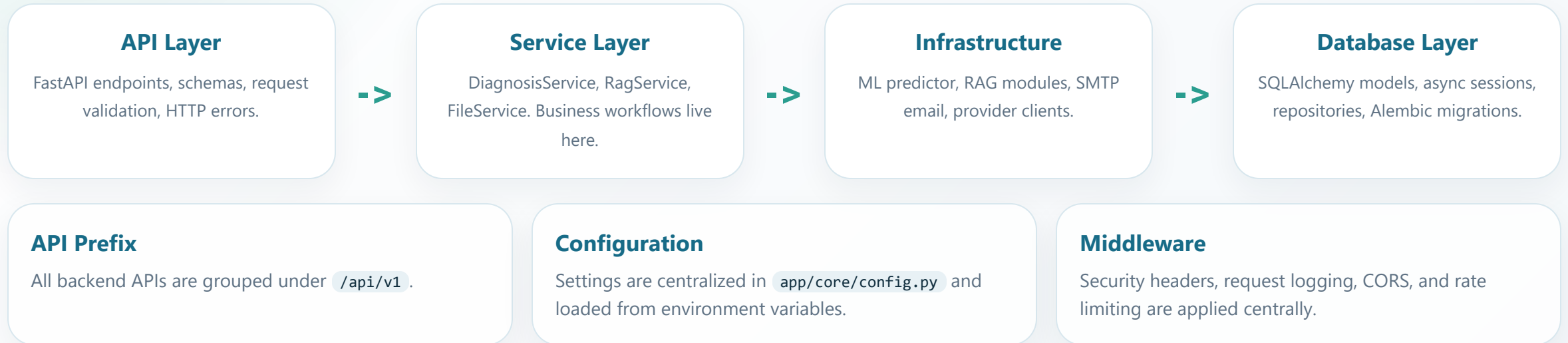
Useful for database calls, file operations, email sending, and RAG workflows without blocking the server.

Why not Django?

Django is powerful but heavier for an API-focused ML backend. FastAPI keeps the backend lighter and more modular.

Architecture / Structure

The backend follows layered clean architecture to separate request handling, business logic, infrastructure, and persistence.



Design Patterns Used

Layered Architecture

Separates endpoints, services, infrastructure, and database code.

Service Layer

Business workflows live in services such as `DiagnosisService` and `RagService`.

Repository Pattern

Repositories hide database details and provide clean data-access methods.

DTO / Mapper Pattern

Schemas and mappers keep response shapes consistent for the frontend.

Provider / Strategy

RAG can switch between Groq, xAI, OpenAI, and local fallback using configuration.

Fallback Pattern

If an LLM fails, the deterministic local generator still returns a safe response.

Dependency Injection

FastAPI dependencies provide current user, DB session, and service instances.

Middleware Pattern

Cross-cutting concerns like logging, rate limits, and security headers are centralized.

Centralized Config

Secrets, URLs, model path, CORS, limits, and providers are controlled by environment variables.

Diagnosis Request Flow

Protected API Flow

- Frontend sends JWT token in Authorization header.
- FastAPI dependency extracts current user ID.
- Upload endpoint validates file type and size.
- Service layer saves file with a safe generated name.
- ML predictor runs inference.
- Repository stores diagnosis record.
- Mapper returns frontend-friendly response.

Main Backend Endpoints

Area	Examples
Health	GET /api/v1/health , GET /api/v1/ready
Auth	/auth/signup , /auth/login , /auth/me
Password Reset	/auth/forgot-password , /auth/reset-password
Diagnosis	POST /diagnosis , GET /history , GET /diagnosis/{id}
Family	GET/POST/PUT/DELETE /members
RAG	POST /rag/chat , POST /rag/final-report

EfficientNet-B4 Prediction Pipeline

Why EfficientNet-B4?

- Good balance between accuracy and model size.
- Strong image feature extraction for medical image classification.
- Works well with transfer learning and PyTorch/timm workflows.

What the Model Returns

- Predicted disease label.
- Confidence score.
- Top predictions.
- Image quality score and quality label.
- Analysis ID for history, result refresh, RAG chat, and reports.

Supported Classes

Urticaria

Tinea Circinata

Annular Lichen / Lichen Planus

Psoriasis

Mycosis Fungoides / MF

Healthy

Important Concept

Model predicts, RAG explains. The ML model classifies the image. The RAG assistant explains the condition using a controlled knowledge base and safety rules.

Database Design

The project uses one application database locally: SQLite. SQLAlchemy async ORM and Alembic migrations keep the schema organized and portable.

Table	Main Columns	Purpose
users	id, email, username, hashed_password, is_active, created_at, updated_at	Stores user accounts and authentication identity.
diagnoses	id, user_id, family_member_id, session_id, image_path, image_quality_score, image_quality_label, disease_type, probability, confidence_percentage, all_predictions, extra_metadata, created_at, updated_at	Stores each diagnosis result and links it to the owner.
family_members	id, user_id, name, relation, date_of_birth, gender, notes, created_at, updated_at	Allows users to run analysis for themselves or family members.
password_reset_tokens	id, user_id, token_hash, expires_at, used_at, created_at	Stores hashed single-use password reset tokens.
alembic_version	version_num	Internal migration tracking table, not business data.

Why SQLite?

Simple local/demo setup with no external database server.

Why Alembic?

Tracks schema changes through migrations instead of manual SQL edits.

Production Note

PostgreSQL is recommended for production scalability and reliability.

RAG Architecture and What We Added

1

Diagnosis Context

Load diagnosis by analysis_id and verify ownership.

2

Intent Detection

Detect diagnosis, treatment, urgent care, contagion, dosage, age group, and compound questions.

3

Knowledge Loading

Load `skin_data1.txt` as the dermatology knowledge base.

4

Chunking

Split the knowledge file into searchable chunks.

5

Local Index

Build a lightweight TF-IDF-like vector index with disease tags.

6

Reranking

Use predicted_label as a strong signal so sources match the selected disease.

7

Generation

Use Groq/xAI/OpenAI if configured, otherwise local deterministic fallback.

8

Validation

Post-generation validator rejects unsafe or inaccurate medical answers.

Medical Guardrails in the Assistant

Deterministic Safety Profiles

- Canonical disease names in English and Arabic.
- Urticaria is forced as not contagious.
- Tinea is forced as contagious.
- Urgent red flags are controlled by backend profiles.

No Personal Prescription

- No exact dose.
- No frequency.
- No treatment duration.
- Strong treatments are only under medical supervision.

Provider Safety

- Groq, xAI, and OpenAI are optional.
- No API keys are exposed in the frontend.
- Out-of-scope questions do not call LLM generation.
- If provider fails, local fallback still answers safely.

Post-generation Validation

- Blocks wrong translations like Urticaria as nausea.
- Rejects false contagion claims for Urticaria.
- Removes unsafe wording and falls back to deterministic answers.

Report System

Preliminary Report

- Generated from the diagnosis result page.
- Based on the AI diagnosis result and general educational guidance.
- Downloadable or printable from the result page.
- Does not call the RAG consultation report endpoint.

Consultation Report

- Generated after the chatbot conversation.
- Uses diagnosis context and summarized patient concerns.
- Includes general care guidance, red flags, and disclaimer.
- Can be printed or copied from the chat report modal.

Medical wording rule

We use "Preliminary Report" and "Consultation Report". We avoid wording like final diagnosis, recommended treatment, or personal prescription.

Frontend Overview

Why React/Vite?

- React supports reusable components and state-driven UI.
- Vite gives fast local development and optimized production builds.
- Good fit for dashboards, forms, modals, chat UI, and multi-language screens.

Frontend Organization

- `pages` : Home, Upload, Result, Chat, History, Dashboard, Auth pages.
- `components` : Navbar, Footer, Hero, disease cards, family manager.
- `services` : auth, diagnosis, chat, family API calls.
- `contexts` : auth, language, and theme state.
- `locales` : Arabic and English UI text.

RTL

Arabic layout and spacing are handled across Home, Result, Upload, and Chat.

Chat

Diagnosis-aware chat reads analysisId from the URL and preserves history per diagnosis.

Reports

Basic report download and consultation report print/copy flows are supported.

API Config

Uses `VITE_API_BASE_URL` and a shared Axios client with JWT handling.

Main Frontend Pages

Home

Project explanation, supported conditions, Arabic/English content, and medical safety wording.

Auth Pages

Sign up, sign in, forgot password, reset password, JWT session handling.

Upload

Image upload, patient selection, crop flow, validation text, and diagnosis submission.

Result

Displays diagnosis, confidence, image quality, top predictions, details, and preliminary report download.

History / Dashboard

Shows previous diagnoses and opens report pages by analysis ID.

Chat

RAG assistant, quick questions, hidden sources, global disclaimer, and consultation report generation.

Security Features

Authentication

JWT auth protects private endpoints and redirects users when a token expires.

Ownership Checks

Users can only access their own diagnoses, RAG chat context, and generated reports.

Password Security

Passwords are hashed. Reset tokens are hashed, expiring, and single-use.

Upload Security

Only JPG, JPEG, PNG are accepted. Max size is 10MB. Real image content is verified.

Rate Limiting

Sensitive routes such as login, signup, reset, diagnosis, and RAG chat are rate limited.

Secrets Hygiene

LLM and SMTP keys are backend-only. `.env` is ignored by Git. Examples use placeholders only.

Docker and GitHub Readiness

Docker Setup

- Backend Dockerfile runs FastAPI on port 8000.
- Frontend Dockerfile builds Vite and serves with nginx.
- `docker-compose.yml` runs backend and frontend together.
- Persistent folders can be mounted for SQLite, uploads, and RAG index cache.

GitHub Safety

- `.gitignore` and `.dockerignore` exclude secrets and generated files.
- No real API keys or SMTP passwords are committed.
- README files describe local and Docker setup.
- Model weights are around 67.7 MiB, so Git LFS is recommended for a polished public repository.

Deployment note

For production, use a strong `SECRET_KEY`, `AUTH_REQUIRED=true`, strict CORS origins, backend-only LLM keys, and PostgreSQL instead of SQLite.

Medical Safety and Limitations

What the System Can Do

- Provide a preliminary AI-assisted prediction.
- Explain supported conditions using a local knowledge base.
- Highlight red flags and when to seek urgent care.
- Generate educational reports for consultation preparation.

What the System Cannot Do

- It cannot replace a dermatologist.
- It cannot provide a final medical diagnosis.
- It cannot prescribe personal medication.
- It cannot decide exact dose, frequency, or duration.

Core disclaimer

DermaVision is an educational and assistive system. A dermatologist must confirm the diagnosis and decide the correct treatment.

What Makes DermaVision Strong

Technical Strength

End-to-end full-stack system with React/Vite, FastAPI, SQLAlchemy, Alembic, EfficientNet-B4, RAG, Docker, and provider-based LLM integration.

Product Strength

Clear user journey: upload image, see diagnosis, ask assistant, generate reports, and view history.

Architecture Strength

Clean layered backend with service, repository, mapper, provider, middleware, and fallback patterns.

Safety Strength

Medical guardrails prevent personal prescriptions, false contagion claims, unsafe LLM answers, and unsupported medical certainty.

DermaVision is not just an AI classifier. It is a complete, secure, bilingual, medically cautious dermatology assistant workflow.